UNIVERSIDAD NACIONAL DE COLOMBIA

Facultad de Ingeniería y Arquitectura

Movimiento, Velocidad y Trayectorias en ROS2

Informe de Laboratorio – Capítulo 3 (Corke)

Curso: Robótica
Código: MAN-4101136-1-2026-03

Integrantes:

Lina Robsana Jamioy Tisoy
Tomás Rocha Castaño

Profesor:
Jaime Enrique Arango Castro

29 de septiembre de 2026

Índice

1. Objetivos **2**

2. Entorno de Trabajo **2**

3. PARTE 1: Velocidad y Movimiento en el Tiempo **3**
3.1. Actividad 1: Integración de orientación con Euler y Cuaterniones 3

4. PARTE 2 y 3: ROS2 – Integración de IMU y Simulación **6**
4.1. Actividad 2 y 3: Nodos Simulator e Integrator 6

5. PARTE 4: Trayectorias en el Tiempo **9**
5.1. Actividades 4 y 5: Perfiles Quíntico y Trapezoidal 9

6. PARTE 5: Interpolación de Orientación **11**
6.1. Actividad 6: Interpolación Esférica (SLERP) 11

7. PARTE 6: Visualización en RViz2 **12**
7.1. Actividad 7: Configuración y Visualización de Frames 12

8. Discusión Final **13**

9. Conclusiones **13**

1. Objetivos

1. Comprender la relación matemática y práctica entre la velocidad angular y el cambio de orientación.
2. Implementar integración de orientación en el tiempo empleando ángulos de Euler y cuaterniones.
3. Desarrollar nodos en ROS2 para simular una IMU e integrar sus datos publicando transformaciones mediante TF2.
4. Generar y comparar perfiles de trayectorias temporales (Quíntica vs. Trapezoidal) con `robotics-toolbox-python`.
5. Aplicar interpolación esférica de orientación (`SLERP`) entre dos poses objetivo.
6. Visualizar transformaciones dinámicas en RViz2.

2. Entorno de Trabajo

- **Sistema Operativo:** Ubuntu 24.04 LTS
- **Distribución ROS2:** Jazzy Jalisco
- **Lenguaje de Programación:** Python 3.12
- **Librerías principales:** `rclpy`, `sensor_msgs`, `geometry_msgs`, `tf2_ros`, `SciPy`, `NumPy`, `roboticstoolbox-python`.

3. PARTE 1: Velocidad y Movimiento en el Tiempo

3.1. Actividad 1: Integración de orientación con Euler y Cuaterniones

Se estudió cómo la pose cambia dinámicamente mediante integración discreta en el tiempo usando una velocidad angular constante $\boldsymbol{\omega} = [0, 0, 1]^T$ rad/s y un paso de tiempo $\Delta t = 0{,}01$ s.

```python
import numpy as np
from scipy.spatial.transform import Rotation as R

dt = 0.01
w = np.array([0, 0, 1])
r = R.identity()

for i in range(100):
    r = r * R.from_rotvec(w * dt)

print(" ngulos  de Euler (grados):", r.as_euler('xyz', degrees=True))
print("Cuaterni n:", r.as_quat())
```

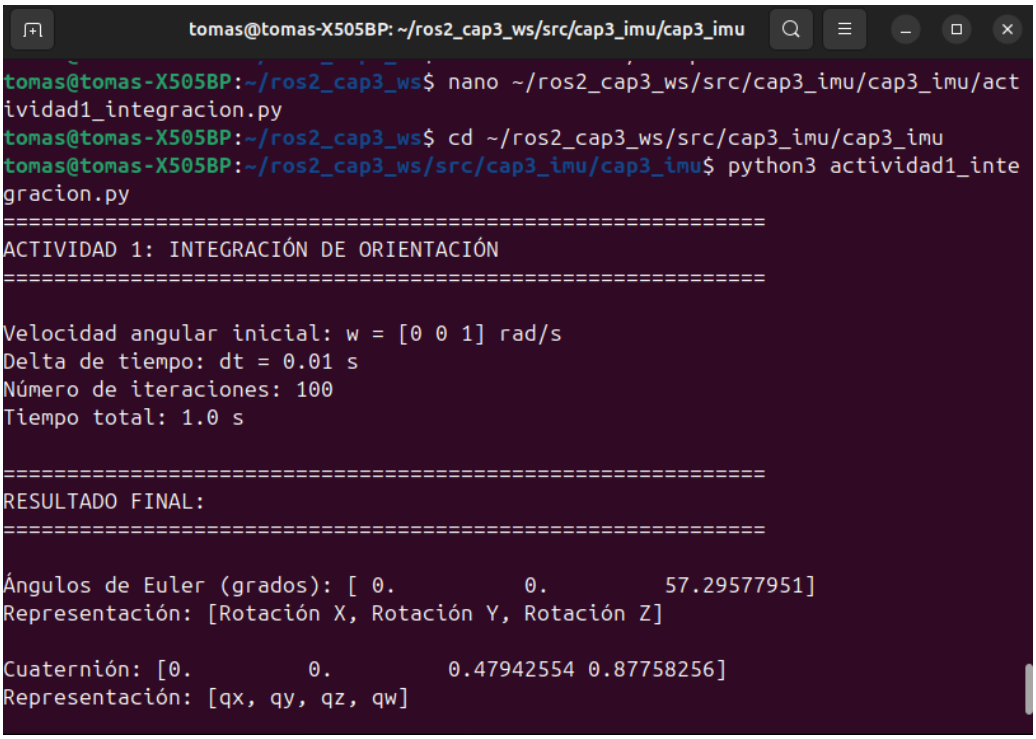


Figura 1: Resultados de la integración de orientación en el eje Z expresados en Euler y Cuaternión.

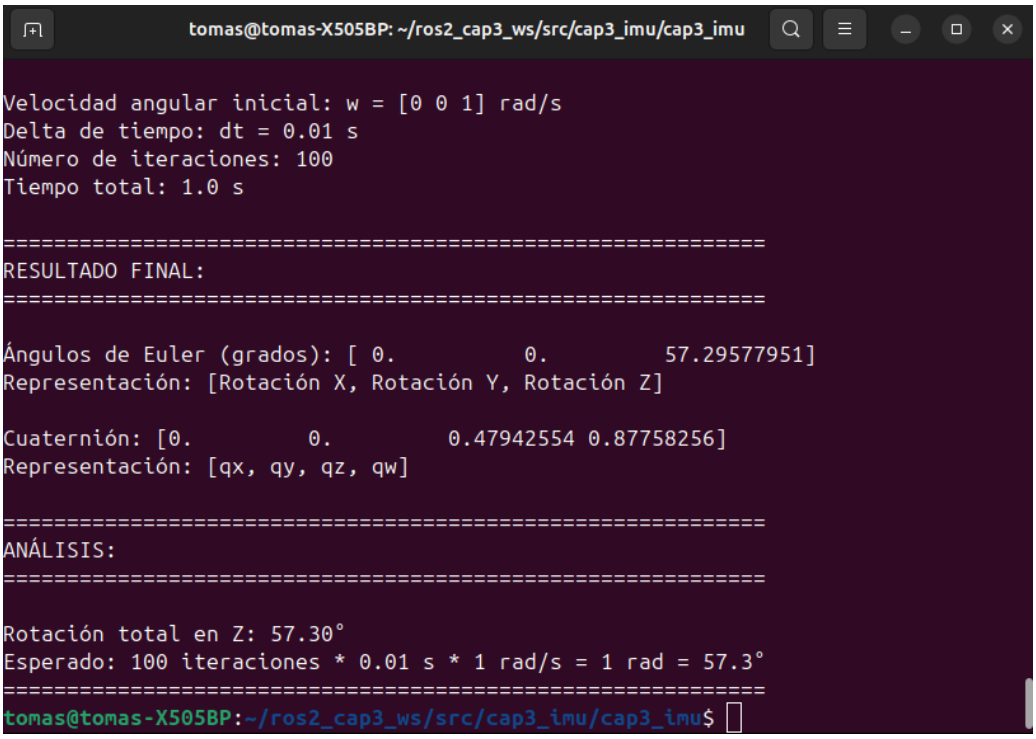


Figura 2: Análisis de resultados: comparación entre el ángulo esperado (57,3°) y el integrado.

Posteriormente, se realizó la integración usando cuaterniones tipo IMU para velocidades angulares simultáneas en los tres ejes $\boldsymbol{\omega} = [0{,}1, 0{,}2, 0{,}3]^T$ rad/s durante 200 iteraciones.

```python
from scipy.spatial.transform import Rotation as R
import numpy as np

q = R.identity()
dt = 0.01
w = [0.1, 0.2, 0.3]

for i in range(200):
    dq = R.from_rotvec(np.array(w) * dt)
    q = q * dq

print("Cuaterni n final [qx, qy, qz, qw]:", q.as_quat())
print(" ngulos de Euler equivalentes (deg):", q.as_euler('xyz', degrees=True))
```

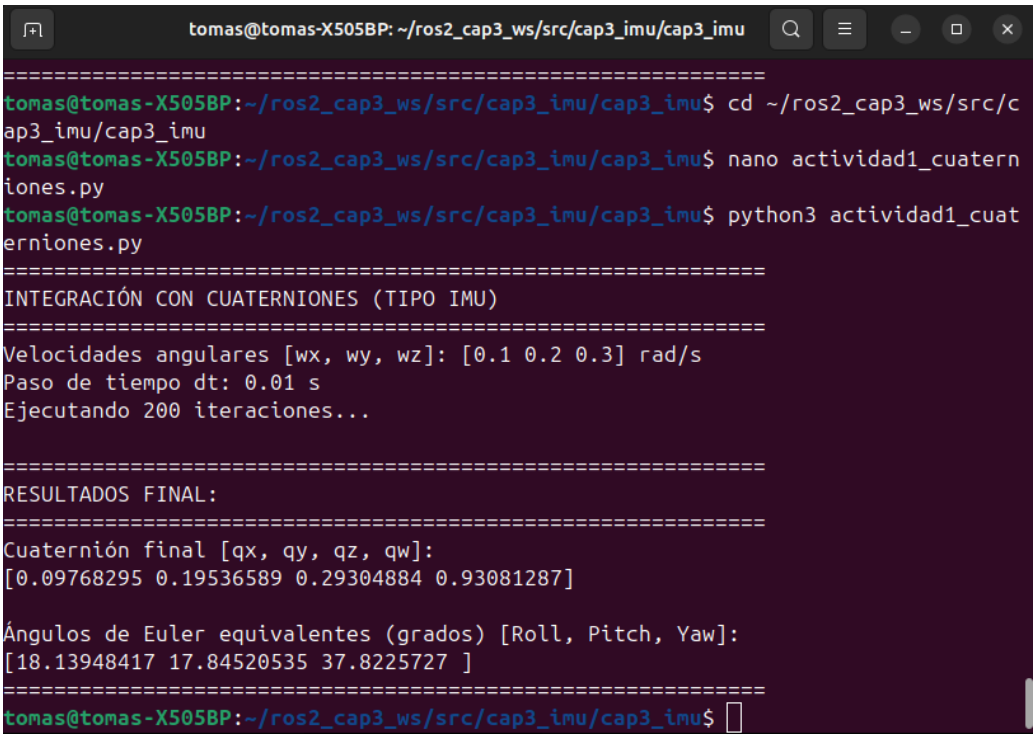


Figura 3: Salida de la integración con cuaterniones para tres ejes acoplados.

Preguntas y Análisis

1. **¿Qué representa ω?** Representa el vector de velocidad angular instantánea $[w_x, w_y, w_z]^T$, indicando la dirección del eje de rotación y la rapidez angular en radianes por segundo.

2. **¿Qué operación se está realizando sobre la orientación?** Se realiza una composición sucesiva de rotaciones relativas mediante la multiplicación de matrices/cuaterniones $q_{k+1} = q_k \cdot \Delta q$, acumulando pequeños incrementos de rotación $\Delta q = \text{Rot}(\boldsymbol{\omega} \cdot \Delta t)$.

4. PARTE 2 y 3: ROS2 – Integración de IMU y Simulación

4.1. Actividad 2 y 3: Nodos Simulator e Integrator

Se creó la estructura del espacio de trabajo `/ros2_cap3_ws` y el paquete `cap3_imu`. Se construyeron dos nodos:

1. `imu_simulator.py`: Publica datos periódicos de una IMU en el tópico `/imu` a 100 Hz con $\omega_z = 0{,}5$ rad/s.

2. `imu_integrator.py`: Se suscribe a `/imu`, integra numéricamente la velocidad angular acumulando un cuaternión y transmite la transformada `TF2` entre los marcos `world` e `imu_link`.

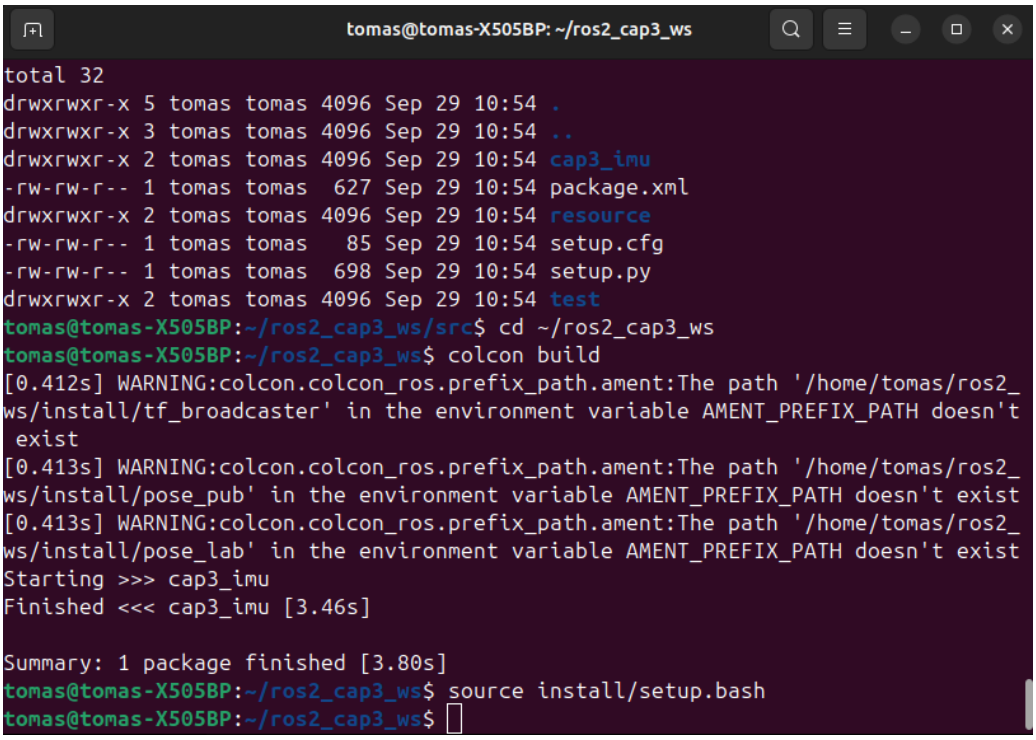


Figura 4: Creación del workspace y primer proceso de compilación del paquete `cap3_imu`.

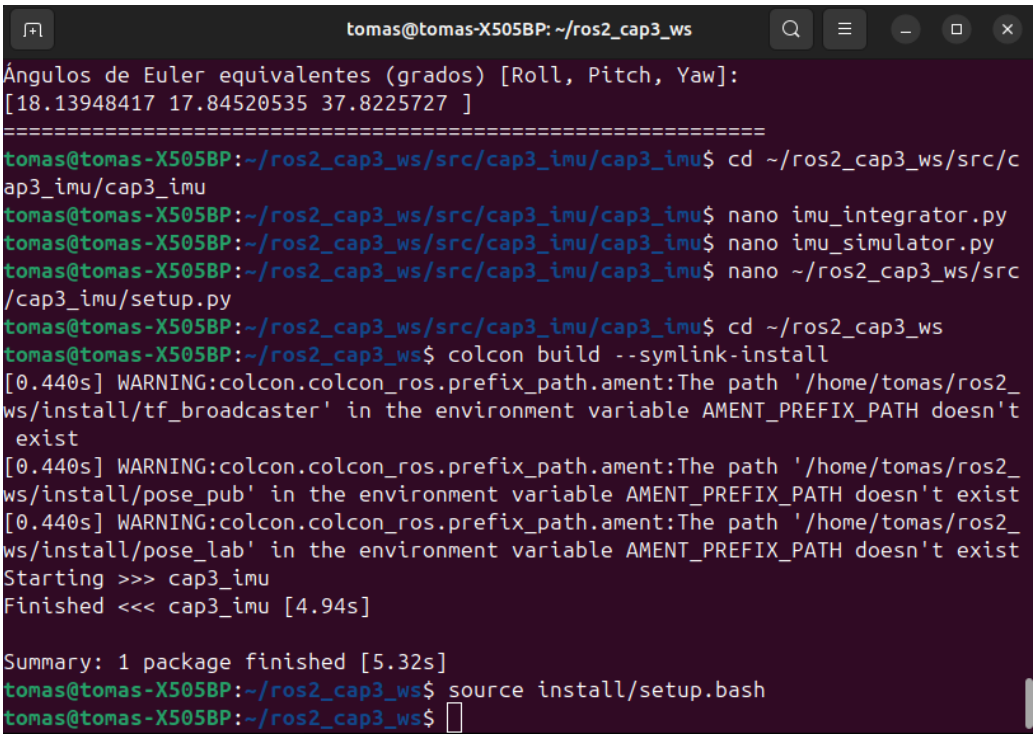


Figura 5: Compilación exitosa con `colcon build -symlink-install`.

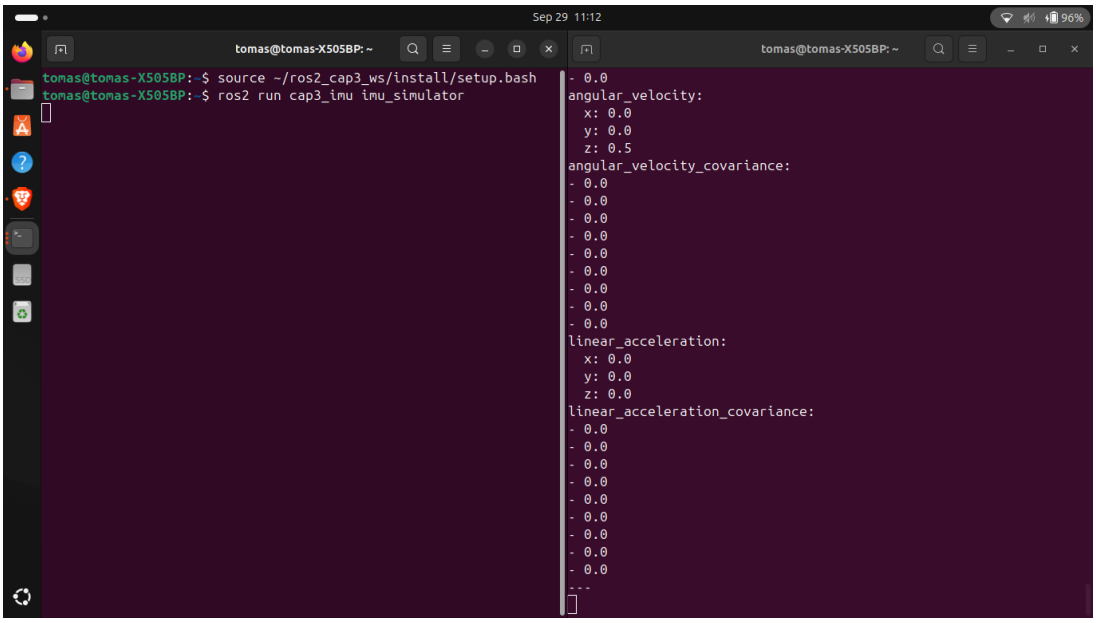


Figura 6: Ejecución de `imu_simulator` y verificación en terminal con `ros2 topic echo /imu`.

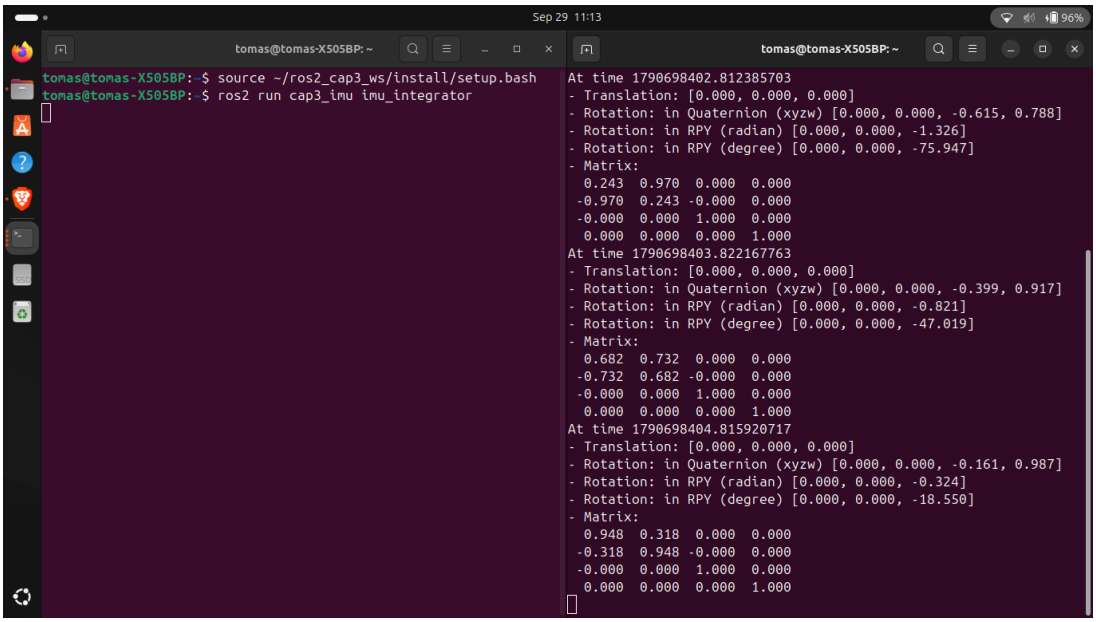


Figura 7: Ejecución de `imu_integrator` y verificación del árbol TF2 con `ros2 run tf2_ros tf_echo world imu_link`.

Pregunta

¿Qué ocurre si cambia el eje de rotación? Si la velocidad angular deja de ser pura en Z y se aplican componentes en X o Y, el vector resultante de rotación cambia su orientación espacial continua. En Euler se evidencia acoplamiento entre Roll, Pitch y Yaw; en cuaterniones, los cuatro componentes $[q_x, q_y, q_z, q_w]$ varían dinámicamente manteniendo $\|q\| = 1$.

5. PARTE 4: Trayectorias en el Tiempo

5.1. Actividades 4 y 5: Perfiles Quíntico y Trapezoidal

Se implementaron y compararon dos tipos de trayectorias utilizando la librería `roboticstoolbox-python`:

- **Trayectoria Quíntica** (`quintic`)**:** Polinomio de orden 5 que garantiza condiciones de borde suaves en posición, velocidad y aceleración $(q, \dot{q}, \ddot{q})$.
- **Trayectoria Trapezoidal** (`trapezoidal`)**:** Perfil con aceleración en bloques constantes, velocidad de crucero plana y desaceleración constante.

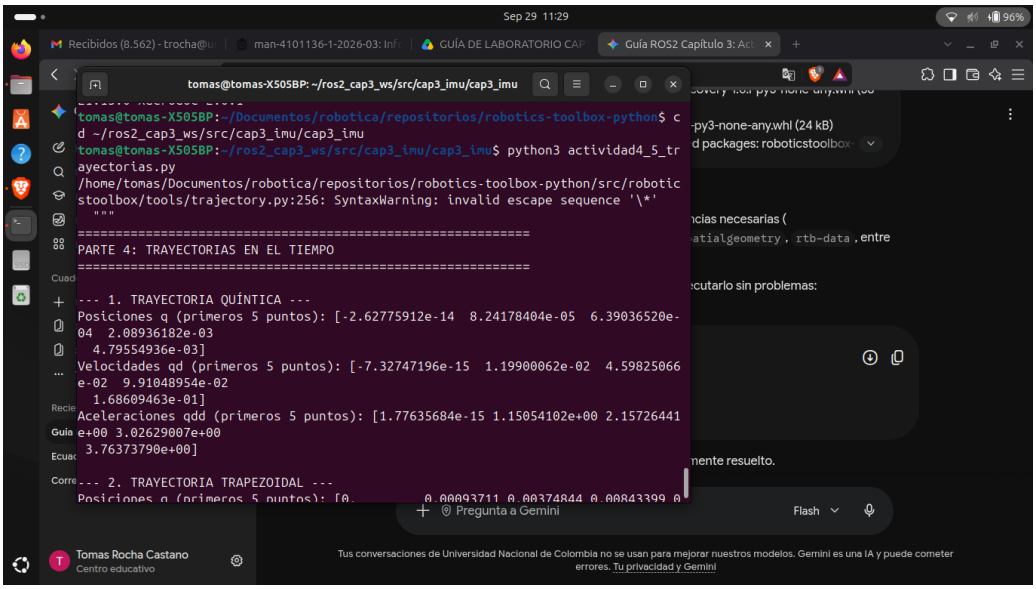


Figura 8: Salida del script mostrando los primeros valores de posición, velocidad y aceleración.

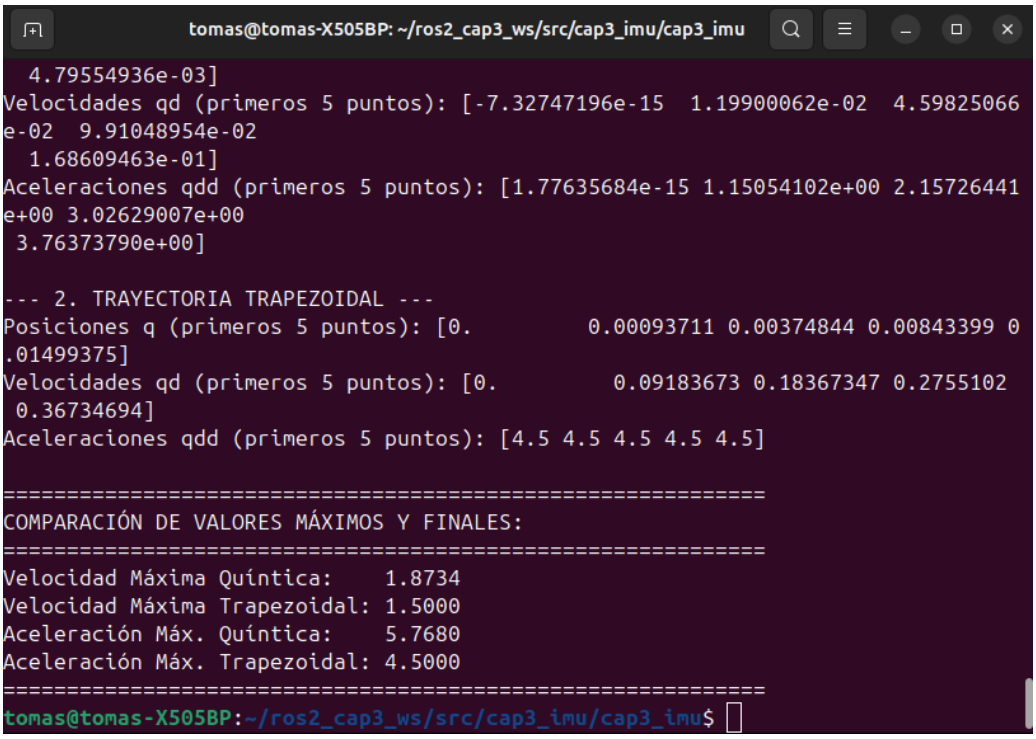


Figura 9: Comparación cuantitativa de valores máximos alcanzados por cada trayectoria.

Cuadro Comparativo de Resultados

Métrica	Trayectoria Quíntica	Trayectoria Trapezoidal
Velocidad Máxima ($\dot{q}_{mx}$)	1,8734 rad/s	1,5000 rad/s
Aceleración Máxima ($\ddot{q}_{mx}$)	5,7680 rad/s^2	4,5000 rad/s^2
Continuidad en Aceleración	Continua (Cero en extremos)	Discontinua (Saltos en escalón)

Preguntas

1. **¿Qué diferencias observa entre las trayectorias?** La quíntica requiere picos de velocidad y aceleración más elevados para garantizar la continuidad del jerk (tirones), mientras que la trapezoidal optimiza los picos de velocidad imponiendo aceleraciones constantes.

2. **¿Qué criterio permite seleccionar una trayectoria?** Si la prioridad es minimizar el desgaste mecánico, vibraciones y esfuerzos en la estructura (p. ej., robots de precisión o transporte de carga frágil), se selecciona la **quíntica**. Si se busca minimizar la potencia consumida en el motor operando a velocidad constante la mayor parte del tiempo, se elige la **trapezoidal**.

6. PARTE 5: Interpolación de Orientación

6.1. Actividad 6: Interpolación Esférica (SLERP)

Se implementó el algoritmo SLERP (*Spherical Linear Interpolation*) para realizar una transición suave entre dos orientaciones (0° y 90° sobre Z).

```python
import numpy as np
from scipy.spatial.transform import Rotation as R, Slerp

r1 = R.from_euler('z', 0, degrees=True)
r2 = R.from_euler('z', 90, degrees=True)

slerp = Slerp([0, 1], R.concatenate([r1, r2]))
times = np.linspace(0, 1, 5)
interp = slerp(times)

print(" ngulos  Euler (xyz deg):")
print(interp.as_euler('xyz', degrees=True))
```

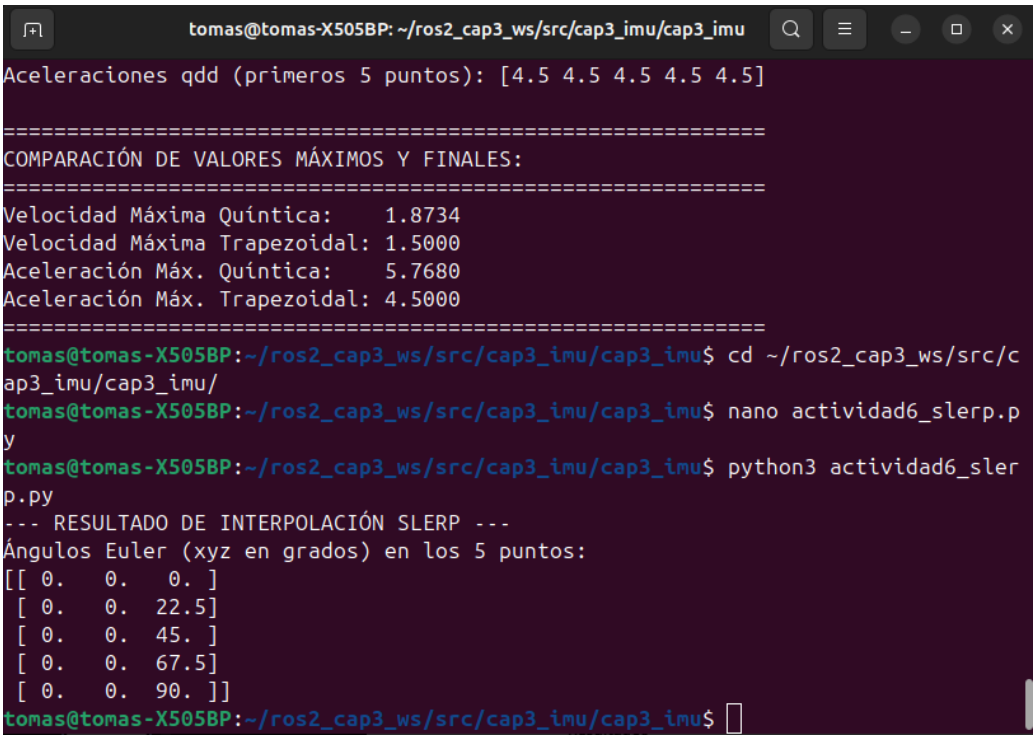


Figura 10: Ejecución de la interpolación de orientación SLERP generando 5 puntos equidistantes.

Pregunta

¿Qué características debe cumplir una interpolación de orientación? Debe ofrecer velocidad angular constante durante la transición, seguir el camino geodésico más corto sobre la esfera unitaria de cuaterniones S^3, carecer de singularidades (*gimbal lock*) y ser independiente del sistema de coordenadas elegido.

7. PARTE 6: Visualización en RViz2

7.1. Actividad 7: Configuración y Visualización de Frames

Se lanzó la herramienta RViz2 con el marco de referencia fijo `world`. Se añadieron las vistas de `TF` y `Axes` para seguir en tiempo real la rotación continua del sistema de coordenadas `imu_link` generada por la integración discreta.

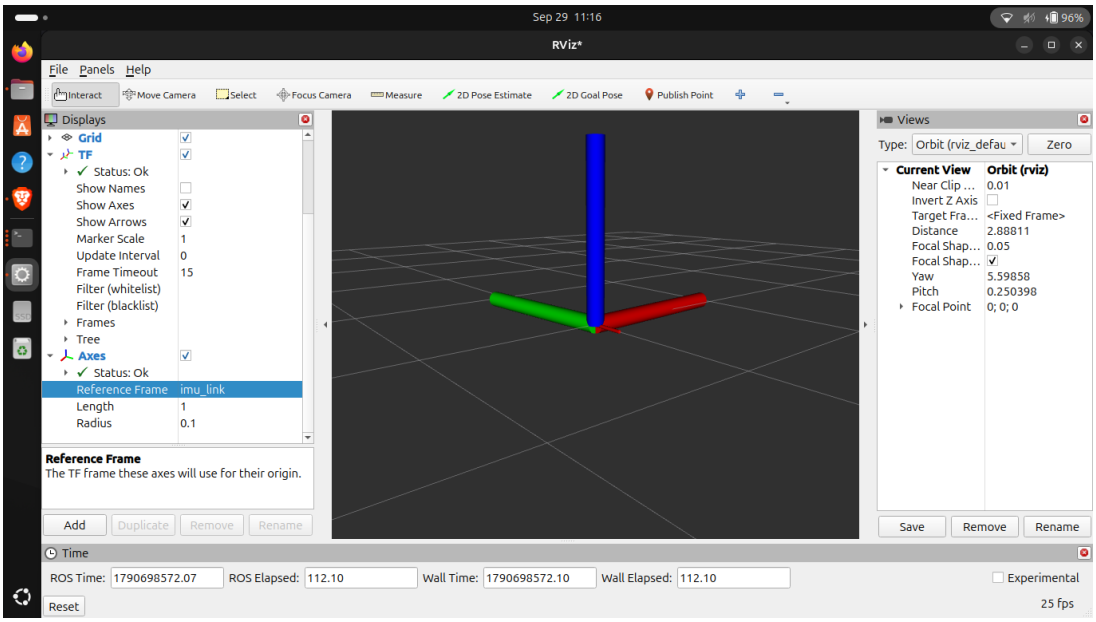


Figura 11: Entorno RViz2 mostrando los ejes del sistema `imu_link` rotando dinámicamente respecto a `world`.

8. Discusión Final

1. **¿Qué relación existe entre velocidad angular y orientación?** La velocidad angular $\boldsymbol{\omega}$ es la derivada temporal de la orientación. Integrar $\boldsymbol{\omega}$ en un intervalo de tiempo da como resultado la variación acumulada de pose del cuerpo rígido.

2. **¿Qué limitaciones tiene la integración discreta?** Sufre de acumulación progresiva de error (*drift* o deriva) por aproximación de primer orden, es sensible al ruido del sensor y puede perder precisión si el paso de integración Δt es muy grande respecto a la dinámica del movimiento.

3. **¿Qué ventajas tiene el uso de cuaterniones?** Evitan el bloqueo de cardán (*Gimbal Lock*), reducen el número de parámetros frente a matrices 3×3 (4 vs 9), requieren menor esfuerzo computacional y permiten interpolación lineal esférica (`SLERP`).

4. **¿Qué diferencias existen entre trayectoria y velocidad?** La trayectoria especifica la evolución completa de la posición y orientación en función del tiempo $q(t)$, mientras que la velocidad $\dot{q}(t)$ describe únicamente la tasa instantánea de cambio del estado.

5. **¿Cómo se relaciona este laboratorio con aplicaciones reales?** Constituye el pilar básico del funcionamiento de las unidades de medición inercial (IMUs) en navegación inercial (AHRS) para drones, robótica móvil, vehículos autónomos y dispositivos móviles.

6. **¿Por qué integrar con cuaterniones y no matrices?** Las matrices 3×3 pierden la condición de ortonormalidad ($\mathbf{R}^T\mathbf{R} = \mathbf{I}$) debido al redondeo numérico, exigiendo procesos costosos de re-ortonormalización. Los cuaterniones solo requieren una normalización dividir por su norma $\|q\| = 1$.

7. **¿Qué errores introduce la integración discreta?** Errores de truncamiento al aproximar curvas continuas mediante segmentos lineales (Δt), magnificación de ruido blanco del giroscopio y acumulación de error de sesgo (*bias drift*).

8. **¿Por qué la trapezoidal es más usada en la industria?** Porque permite maximizar el tiempo de desplazamiento a la máxima velocidad operacional permitida por los actuadores, optimizando el ciclo de trabajo de los motores en controladores industriales y PLC.

9. Conclusiones

1. La integración continua basada en cuaterniones demostró ser numéricamente más eficiente y robusta frente al acoplamiento de ejes espaciales que las representaciones tradicionales en ángulos de Euler.

2. Se logró articular la arquitectura modular de ROS2, validando cómo un nodo simulador transmite tópicos estándar de IMU y cómo un nodo integrador traduce esos mensajes en transformaciones espaciales continuas (`TF2`) representables en RViz2.

3. La selección de perfiles de velocidad/trayectoria refleja una divergencia clave en ingeniería: minimizar tirones y vibraciones (quíntica) vs. maximizar la eficiencia y velocidad en régimen permanente (trapezoidal).

Apéndice: Evidencia en Video

Se grabó y procesó el archivo de pantalla `06_visualizacion_rviz2.webm` donde se demuestra el funcionamiento simultáneo de los nodos en ROS2 y la rotación en tiempo real del eje `imu_link` sobre RViz2.

Enlace al video de demostración en YouTube:

https://youtu.be/LusyeibyE-I